K-Auto KG 실습
정답과 예상 출력

강사용 워크북 — 판정 기준 · 실측 출력 · 모범답안

환경 판정

↓

적재 검증

↓

탐색 정답

↓

MCP 판정

이 워크북의 값은 네 가지 출처로 나뉩니다

각 슬라이드 오른쪽 위의 태그가 그 쪽 값의 출처다 — 수강생 화면과 대조할 때 먼저 태그를 본다

실측	CSV 계산	코드 확인	예시
실제로 실행	**시드로 계산**	**스크립트 원문**	**형식만 모범**
테스트 · HTML	정상이면 일치	print · 오류	PC마다 다름

※ Docker·Neo4j 실행과 MCP 호출은 이 제작 환경에서 하지 않았다 — 'CSV 계산'·'예시'로 표시

DB 결과 = import_neo4j.py의 MERGE 규칙을 시드 CSV에 그대로 적용해 계산한 값 (출처 검토일 2026-10-19 시드 데이터)

수강생 값이 'CSV 계산'과 다르면 D6 진단부터 시작합니다

판정 기준은 실습 순서와 같은 네 구간으로 묶었습니다

본편 덱의 P0~P14 · C1~C8 · D1~D6 · 과제 A~C 번호를 그대로 쓴다

01	환경 · 설치 판정	**P0 ~ P3** 모범 보고 · 출력 패턴 · .env · 인증 실패
02	데이터 · 적재 판정	**P4 ~ P7 · C1 · C2** 테스트 6건 · 매핑표 · 154 / 299 · verify
03	탐색 · 관계 모범답안	**P8 ~ P10 · C3 ~ C8** HTML 실측 · 기대 결과 · 2홉 · 과제 A·B·C
04	MCP · 진단 · 채점	**P11 ~ P14 · D1 ~ D6** mcp list · read-only · 함정 두 개 · 평가

01

환경 · 설치 판정

P0 ~ P3 — 수강생 보고가 '근거 있는 판정'인지를 본다

예시

P0의 모범 보고는 판정마다 근거 명령이 붙어 있습니다

Windows 수강생의 예 — 값은 PC마다 다르고, 표의 형식이 정답이다

P0 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 | P11 | P12 | P13 | P14

항목	판정	근거 명령 · 출력
OS · 셸	준비됨	Windows 11 · PowerShell
Python · Claude Code	준비됨	py --version · claude --version
Docker / Compose	확인 불가	compose ps 실패 — 엔진 미실행
포트 · Neo4j · MCP	미설치	7474·7687 비어 있음 · MCP 0개

'확인 불가'를 '미설치'로 뭉개지 않았는지가 첫 채점 포인트입니다

나쁜 응답은 근거 없이 '설치됨'을 적고 바로 설치를 시작합니다

P0는 조사 전용 — 설치·설정 변경이 한 줄이라도 있으면 다시 보낸다

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 P10 P11 P12 P13 P14

다시 보낼 응답 — 근거 없는 표

- 출력 없이 '설치됨'
- 진단 중 설치 시작
- 세 칸 구분 없음

P0를 다시 보낸다

통과 응답 — 근거 있는 표

- 명령 출력을 인용
- 설치·변경 없음
- 준비·미설치·확인불가

P1로 넘어간다

진단 단계의 설치 시도는 승인 경계 위반입니다

예시

T1 출력 네 가지 패턴만 알면 다음 지시가 정해집니다

오류 문구는 OS·버전마다 조금씩 다르다 — 핵심 단어로 판별한다

P0 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 | P11 | P12 | P13 | P14

보이는 출력	뜻	다음 지시
claude: command not found	미설치 또는 PATH	D1
Cannot connect to the Docker daemon	CLI는 있고 엔진 꺼짐	D2 · Desktop 실행
py / python3를 찾을 수 없음	Python 미설치	P1 공식 설치
No MCP servers configured	정상 — 아직 미등록	P11에서 등록

마지막 줄은 오류가 아니라 정상입니다 — 수강생이 자주 놀라는 지점입니다

코드 확인

P2의 정답 세 값은 .env.example에 이미 적혀 있습니다

수강생이 바꾸는 값은 NEO4J_PASSWORD 하나뿐이다

P0 P1 **P2** P3 P4 P5 P6 P7 P8 P9 P10 P11 P12 P13 P14

.env.example (v1.2 · 발췌)

```
# Password must be at least 8 characters
NEO4J_URI=bolt://127.0.0.1:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=CHANGE_TO_A_LONG_UNIQUE_PASSWORD
NEO4J_DATABASE=neo4j
NEO4J_READ_ONLY=true
NEO4J_TELEMETRY=false
```

정답

URI bolt://127.0.0.1:7687
USERNAME neo4j
DATABASE neo4j

감점

비밀번호 값을 채팅에 출력

.env가 .gitignore에 들어 있는지도 함께 확인하게 합니다

코드 확인

비밀번호를 안 바꾸면 두 곳에서 서로 다른 오류가 납니다

docker-compose.yml과 scripts/common.py가 각각 비밀번호를 검사한다

P0 P1 P2 **P3** P4 P5 P6 P7 P8 P9 P10 P11 P12 P13 P14

docker compose up -d — .env가 없거나 값이 비었을 때 (핵심 문구)

```
required variable NEO4J_PASSWORD is missing a value:
Set_NEO4J_PASSWORD_in_dotenv
```

Python 스크립트 — 값이 비었거나 CHANGE_TO_로 시작할 때 (common.py 원문)

```
RuntimeError: Set NEO4J_PASSWORD in project .env first.
```

두 오류 모두 정상 동작입니다 — 안내문을 그대로 둔 채 실행했다는 신호입니다

예시

P3 성공 화면은 이름 · 이미지 · 포트 · 상태 네 칸으로 판정합니다

docker compose ps 출력의 예 (열 일부 생략) — docker-compose.yml에 정의된 값과 대조한다

P0 P1 P2 **P3** P4 P5 P6 P7 P8 P9 P10 P11 P12 P13 P14

docker compose ps

```
NAME           IMAGE              STATUS
kauto-neo4j    neo4j:2026.09.0    Up 2 minutes
PORTS: 127.0.0.1:7474->7474/tcp, 127.0.0.1:7687->7687/tcp
```

이름	이미지	포트
kauto-neo4j	**neo4j:2026.09.0**	**127.0.0.1 전용**

Browser 결과 = 1

RETURN 1

첫 실행은 APOC 플러그인을 내려받느라 Browser가 늦게 열릴 수 있습니다

코드 확인

인증 실패의 흔한 원인은 이미 만들어진 볼륨입니다

Neo4j는 볼륨을 처음 만들 때 NEO4J_AUTH로 비밀번호를 정하고, 이후 .env 변경은 반영하지 않는다

P0 P1 P2 **P3** P4 P5 P6 P7 P8 P9 P10 P11 P12 P13 P14

상황

예전에 띄운 볼륨

kauto_neo4j_data

→

증상

.env 비밀번호 변경

Authentication failed

→

안전한 해법

원래 비밀번호 사용

볼륨 삭제는 금지

docker compose down -v는 실습 데이터를 지우므로 강사도 쓰지 않습니다

02

데이터 · 적재 판정

P4 ~ P7 — 테스트 · 매핑표 · 적재 출력 · 검증 출력의 정답

실측

P4의 정답 출력은 테스트 6건 ok와 마지막 OK 한 줄입니다

이 자료 제작 환경에서 실제로 실행한 결과 — 괄호 안 모듈 경로만 생략했다

python -m unittest discover -s tests -v

```
test_capability_to_system_consistency ... ok
test_company_count_unique_ids ... ok
test_company_relationships_schema_and_provenance ... ok
test_no_announced_contract_is_presented_as_executed_supply ... ok
test_relationship_references_and_evidence ... ok
test_stage_links ... ok
----------------------------------------------------------------------
Ran 6 tests in 0.005s

OK
```

이 테스트는 CSV만 검사하며 Neo4j 연결과는 무관합니다

코드 확인

여섯 테스트는 각각 다른 사고를 막습니다

tests/test_dataset.py — FAIL이 나면 어느 CSV를 봐야 하는지 이름으로 알 수 있다

P0 P1 P2 P3 **P4** P5 P6 P7 P8 P9 P10 P11 P12 P13 P14

company_count_unique_ids

기업 30 · ID 중복 없음

companies.csv

references_and_evidence

참조 · 출처 URL

단계·역량 ID 실재

capability_to_system

역량 → 시스템 일관

capabilities.csv

stage_links

단계 링크 유효

자기 자신 연결 금지

schema_and_provenance

관계 타입 · 1차 출처

발표일 ≤ 2026-10-19

no_announced_contract...

계약을 납품으로 안 씀

SUPPLIES_TO 상태 검사

마지막 테스트가 이 실습의 핵심 원칙을 코드로 강제합니다

코드 확인

P5의 노드 매핑표는 다섯 줄로 정리됩니다

import_neo4j.py 기준 — 실제 노드에는 Company · Capability 같은 공용 레이블도 함께 붙는다

노드 레이블	원천 CSV	핵심 열
KAutoCompany	companies.csv	id · name_ko · stage_id
KAutoStage	stages.csv	id · display_order
KAutoCapability · KAutoSystem	capabilities.csv	id · system_id
KAutoSource	두 CSV의 URL 열	source_url · evidence_url
KAutoRelationAssertion	company_relationships.csv	id (rel_001 ~ rel_042)

노드 타입 '6개 이상'은 KAuto 레이블 여섯 개를 말합니다

CSV 계산

관계 타입 매핑은 분류 관계 넷과 근거 관계 넷으로 나뉩니다

건수는 시드 CSV에서 계산 — 기업 간 8종 42건은 별도 표(C1)로 본다

관계 타입	방향	원천	건수
PARTICIPATES_IN_STAGE	기업 → 단계	companies.stage_id	30
HAS_CAPABILITY	기업 → 역량	capability_ids	39
ENABLES_SYSTEM	역량 → 시스템	capabilities.system_id	24
TYPICALLY_FEEDS	단계 → 단계	stage_links.csv	8

Assertion 쪽: ASSERTS_SOURCE · ASSERTS_TARGET · SUPPORTED_BY 각 42건 + HAS_REFERENCE 30건

CSV 계산

현대모비스는 역량이 넷이지만 공급 관계는 0건입니다

P5가 요구한 두 기업 비교의 모범답안 — 시드는 '공식 근거를 확인한 것'만 담는다

현대모비스 — 역량 4 · 관계 4

- 섀시·제동·ADAS·배터리
- 계열 1 · 컨소시엄 2
- JV 계획 1 · 공급 0

시드에 없다 ≠ 사실이 없다

포스코퓨처엠 — 역량 2 · 관계 5

- 양극재 · 음극재
- 공급계약 3 · 기존공급 1
- 계열 1

시드에 있다 = 발표가 있었다

관계가 비어 있는 것을 '거래가 없다'로 읽는 답안은 감점합니다

CSV 계산

두 연습 질문의 정답은 모두 '아니다'이며 근거가 다릅니다

정답만 쓰고 근거 관계 타입이나 쿼리를 밝히지 않은 답은 절반만 인정한다

Q1 · 같은 그룹 = 직접 소유? — 아니다

- AFFILIATED_WITH는 소속만
- 지분은 공시가 따로 필요
- 예: 현대차 – 기아

관계 정의로 답한다

Q2 · 같은 역량 = 납품? — 아니다

- 05 쿼리는 유사성 후보
- LGES · 삼성SDI 공통 역량
- 둘은 오히려 경쟁사

반례로 답한다

같은 '전기차 배터리 셀' 역량을 공유하는 두 회사는 공급망이 아니라 경쟁 관계입니다

CSV 계산

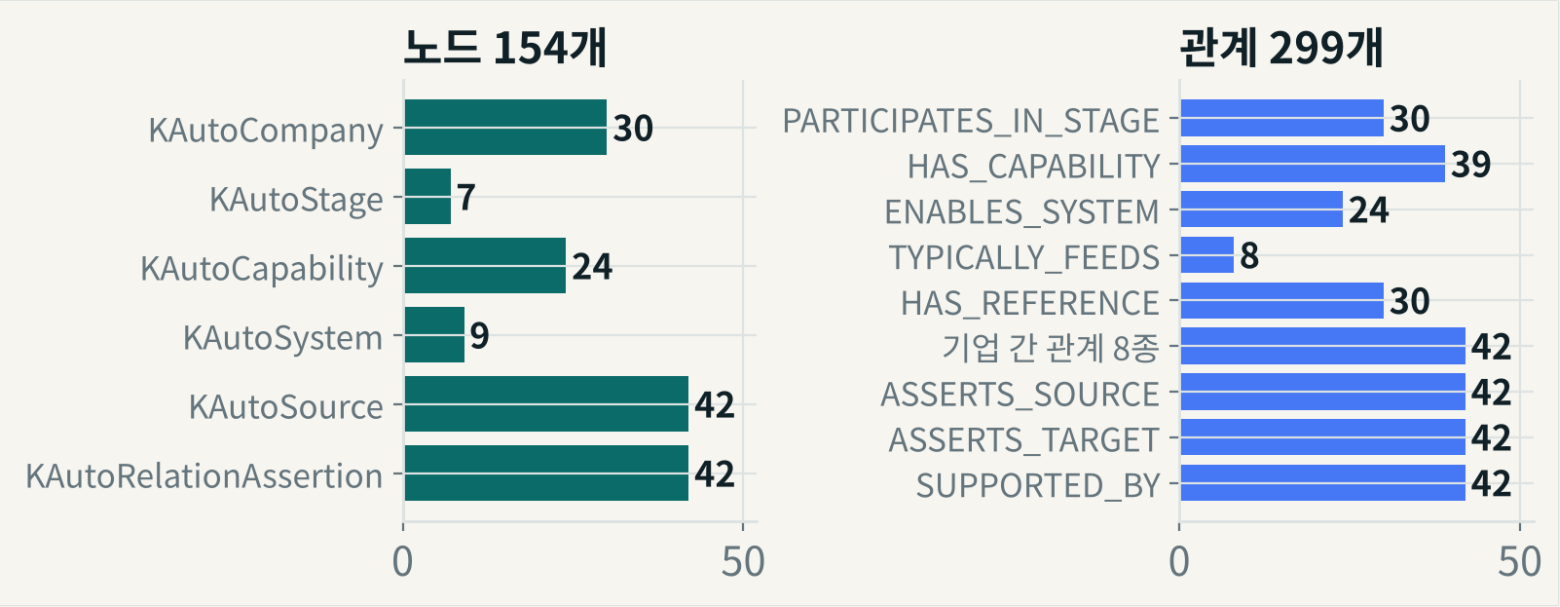
시드 적재는 노드 154개와 관계 299개를 만듭니다

P6 설명의 정답 — 출처 노드 42 = 기업 출처 30 + 관계 출처 URL 12

함께 설명할 것

승인 전 확인

- 127.0.0.1:7687
- 제약조건 6개 먼저
- 삭제 명령 없음



빈 DB가 아니면 기존 데이터 백업 여부를 먼저 물어야 정답입니다

코드 확인

import_neo4j.py가 성공하면 네 줄이 출력됩니다

스크립트의 print 문에서 읽은 출력 — 두 번 실행해도 같은 네 줄과 같은 개수가 나와야 한다

적재 성공 출력

```
Company-to-company relation
  assertions imported: 42
IMPORT COMPLETED (idempotent MERGE,
  no deletions).
Input: 30 companies; 24 capability
  categories; 7 stages.
Evidence: {'official_page_checked': 11,
  'review_required': 19}
```

11 / 19의 뜻

공식 페이지 확인 11곳
추가 검토 19곳

흔한 실패

Set NEO4J_PASSWORD...
→ P2로 돌아간다

실제 출력은 각 줄이 한 줄로 이어져 나옵니다 — 슬라이드 폭에 맞춰 접었습니다

CSV 계산

verify_neo4j.py의 정답 출력은 열 개의 숫자와 PASS 한 줄입니다

기대값은 스크립트가 CSV에서 다시 계산한다 — systems(9)만 비교 없이 출력

verify_neo4j.py 출력 (# 뒤는 강사 메모)

```
companies: 30
stages: 7
capabilities: 24
systems: 9
company_capability_edges: 39        # 기업별 역량 수의 합
source_nodes: 42                    # 기업 출처 30 + 관계 출처 URL 12
company_company_edges: 42
relations_with_provenance: 42
relation_assertions: 42
unsupported_supply_edges: 0
PASS: company-to-company sourced edges verified; unapproved generic SUPPLIES edges = 0.
```

PASS 줄이 없으면 앞의 숫자가 맞아 보여도 FAIL입니다

코드 확인

verify의 AssertionError는 원인을 거의 그대로 말해 줍니다

오류 형식: '<항목>: expected <기대값>, got <실측값>'

P0 P1 P2 P3 P4 P5 P6 **P7** P8 P9 P10 P11 P12 P13 P14

오류 메시지	흔한 원인	조치
companies: expected 30, got 0	다른 DB에 적재	.env의 DATABASE 확인
company_company_edges 초과	이전 버전 데이터	D6 읽기 점검
Unexpected ... SUPPLIES edges	누군가 SUPPLIES 생성	감사 후 승인받아 정리
Set NEO4J_PASSWORD ...	.env 미설정	P2로 돌아감

어떤 경우에도 '전부 지우고 다시 적재'는 정답이 아닙니다

CSV 계산

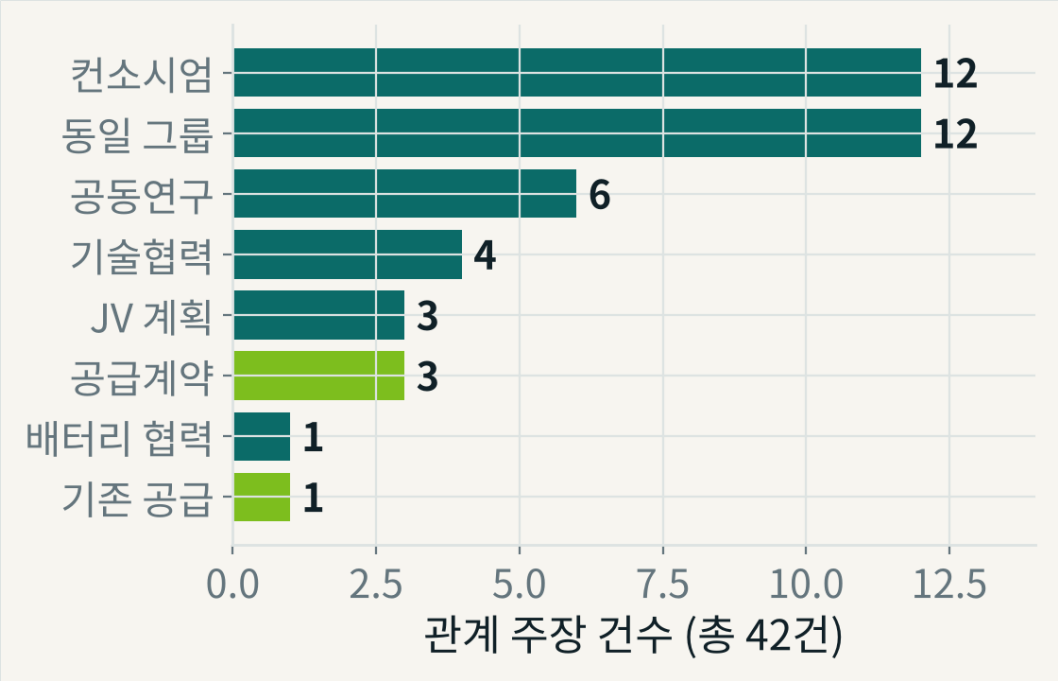
C1의 정답은 여덟 줄이며 합계는 42입니다

ORDER BY relation_count DESC — 12건인 두 타입은 순서가 바뀌어 나올 수 있다

판정

8행 · 합 42

- 12 · 12 · 6 · 4
- 3 · 3 · 1 · 1
- SUPPLIES 없음



같은 건수끼리의 순서 차이는 오답이 아닙니다

CSV 계산

C2는 42행이 나오고 첫 줄은 LG에너지솔루션입니다

ORDER BY 관계, 출발기업 — 문자열은 유니코드 순이라 영문 'LG'가 한글보다 앞선다

P0 P1 P2 P3 P4 P5 P6 **P7** P8 P9 P10 P11 P12 P13 P14

출발기업	관계	도착기업	상태
LG에너지솔루션	AFFILIATED_WITH	LG화학	group_membership
LG에너지솔루션	AFFILIATED_WITH	LG전자	group_membership
삼성전자	AFFILIATED_WITH	삼성SDI	group_membership
포스코	AFFILIATED_WITH	포스코퓨처엠	group_membership
현대자동차	AFFILIATED_WITH	기아 …	group_membership

LIMIT 50이지만 시드가 42건이라 42행에서 멈추는 것이 정답입니다

03

탐색 · 관계 모범답안

P8 ~ P10 · C3 ~ C8 · 과제 A · B · C

실측

render_html.py로 만든 미리보기는 이렇게 열립니다

이 자료 제작 환경에서 실제로 생성해 헤드리스 브라우저로 찍은 화면 — 기본 선택은 현대모비스

확인 포인트

상단 다섯 수치

- 30 · 7 · 24 · 11 · 42
- 단계 칩 7개
- 하단 주의 문구



화면 아래 '실시간 연결되어 있지 않습니다' 문구를 수강생이 읽었는지 묻습니다

실측

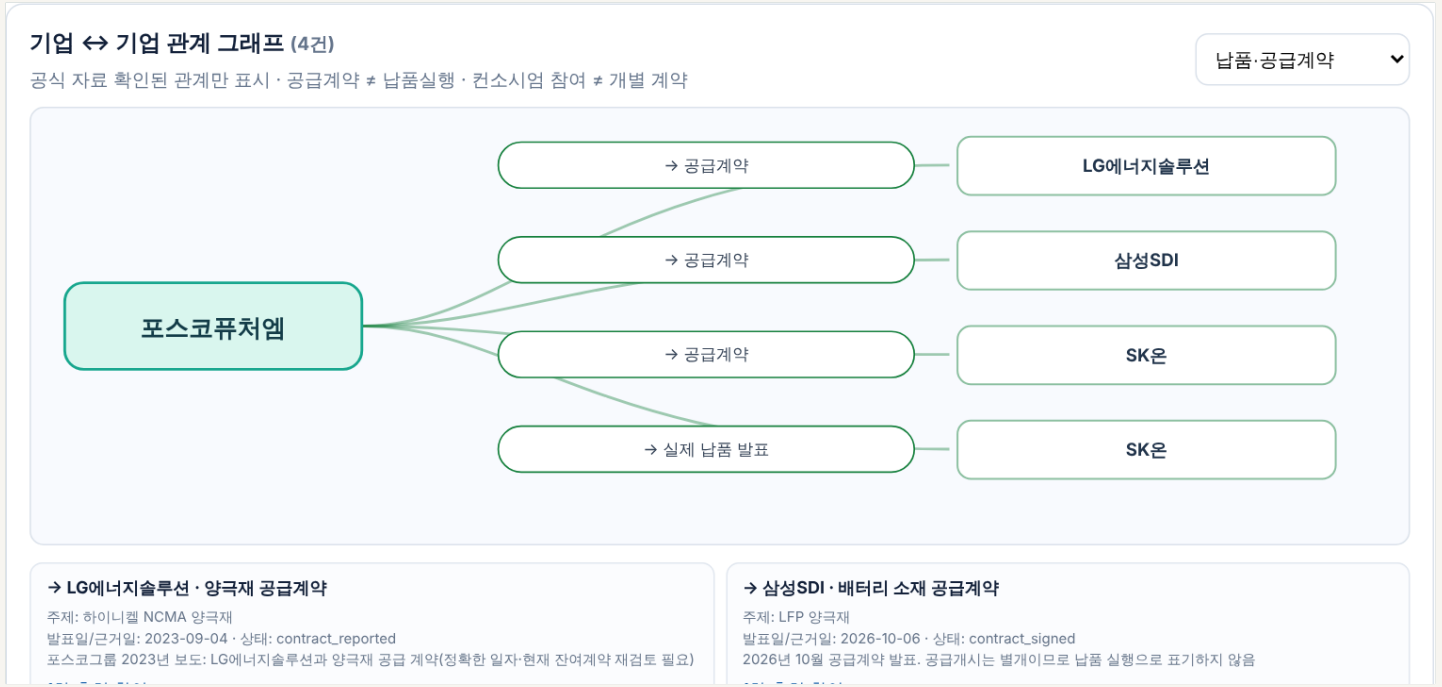
포스코퓨처엠에 '납품·공급계약' 필터를 걸면 4건이 남습니다

기업 검색 → 포스코퓨처엠 선택 → 관계 필터 '납품·공급계약' — 실제 조작 결과

강사 주의

라벨 읽기

- '실제 납품 발표'
- = rel_016 기존 공급
- 현재 물량 아님



화면 라벨이 강해 보여도 근거는 '기존 공급 보도' 한 건입니다

CSV 계산

P9 네 질문의 답은 쿼리 · 결과 형태 · 한계로 정리됩니다

Claude 답안에 '의미하지 않는 것' 칸이 비어 있으면 다시 요청하게 한다

질문	쿼리	기대 결과	의미하지 않는 것
A 밸류체인별 기업 수	01	표 7행 · 합 30	대표 분류일 뿐
B 모비스 → 역량 → 시스템	02	그래프 노드 8	고객사 목록 아님
C 현대차 직접 연결	07	경로 21개	거래 관계 아님
D 공급계약 vs 납품	08	표 4행	납품 물량 아님

출처 확인 방법은 네 질문 모두 r.evidence_url을 RETURN에 넣는 것입니다

CSV 계산

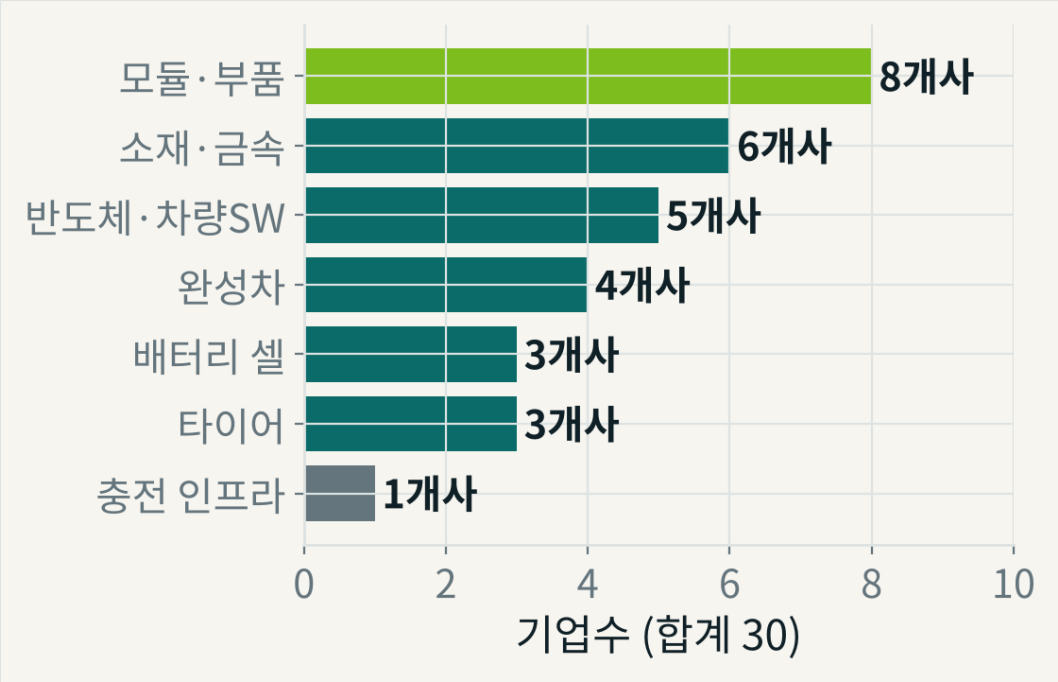
C3은 일곱 행이 나오고 모듈·부품이 8개사로 가장 많습니다

queries/01_industry_overview.cypher — 회사마다 대표 단계 하나만 배정했다

판정

7행 · 합 30

- 최다 모듈·부품 8
- 최소 충전 인프라 1
- 타이어 · 배터리 3



합계가 30이 아니면 적재 누락 또는 다른 DB입니다

CSV 계산

C4는 현대모비스 1 · 역량 4 · 시스템 3의 그래프로 나옵니다

queries/02_mobis_graph.cypher — 제동과 ADAS가 같은 '주행·안전 제어'로 모인다



ENABLES_SYSTEM에는 evidence_class = ontology_concept_not_a_supply_contract가 붙어 있습니다

CSV 계산

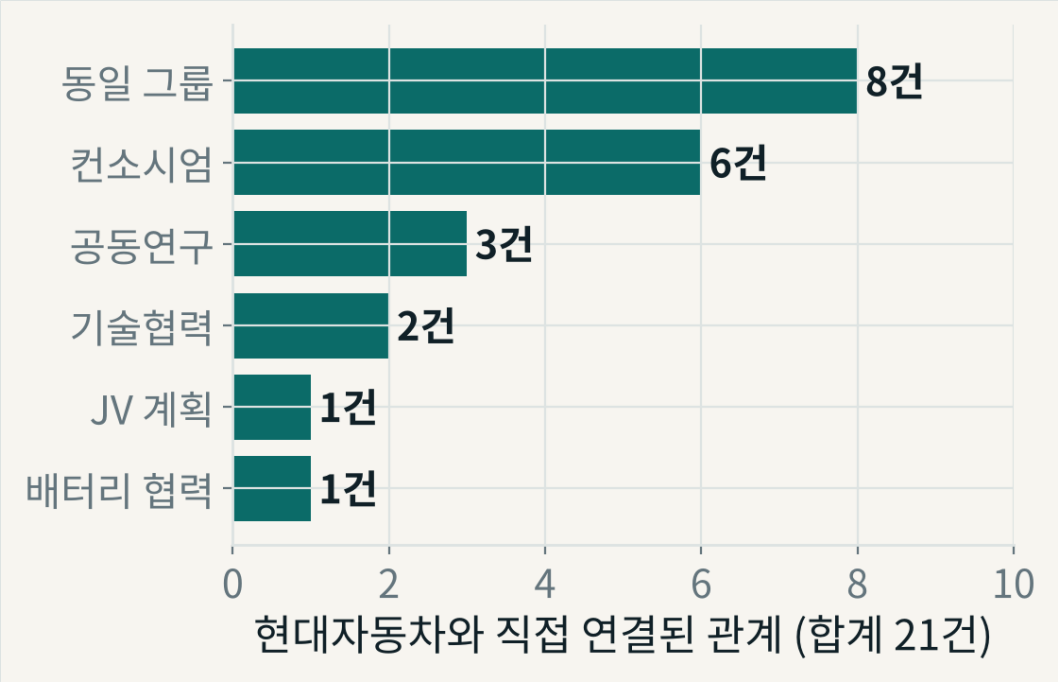
현대자동차는 13개 기업과 21건으로 직접 연결됩니다

queries/07 — 방향을 무시한 패턴 -[r]- 이라 현대차가 도착 쪽인 관계도 함께 잡힌다

판정

경로 21개

- 연결 기업 13곳
- 그룹·컨소 14건
- 공급 관계 0건



현대차와 직접 이어진 공급 관계가 0건이라는 점을 수강생이 짚으면 우수 답안입니다

CSV 계산

공급 관계 표는 포스코퓨처엠에서 출발하는 네 줄뿐입니다

queries/08_supplier_network.cypher — C6 집계 SUPPLIES_TO 1 · SUPPLY_AGREEMENT_WITH 3과 일치해야 한다

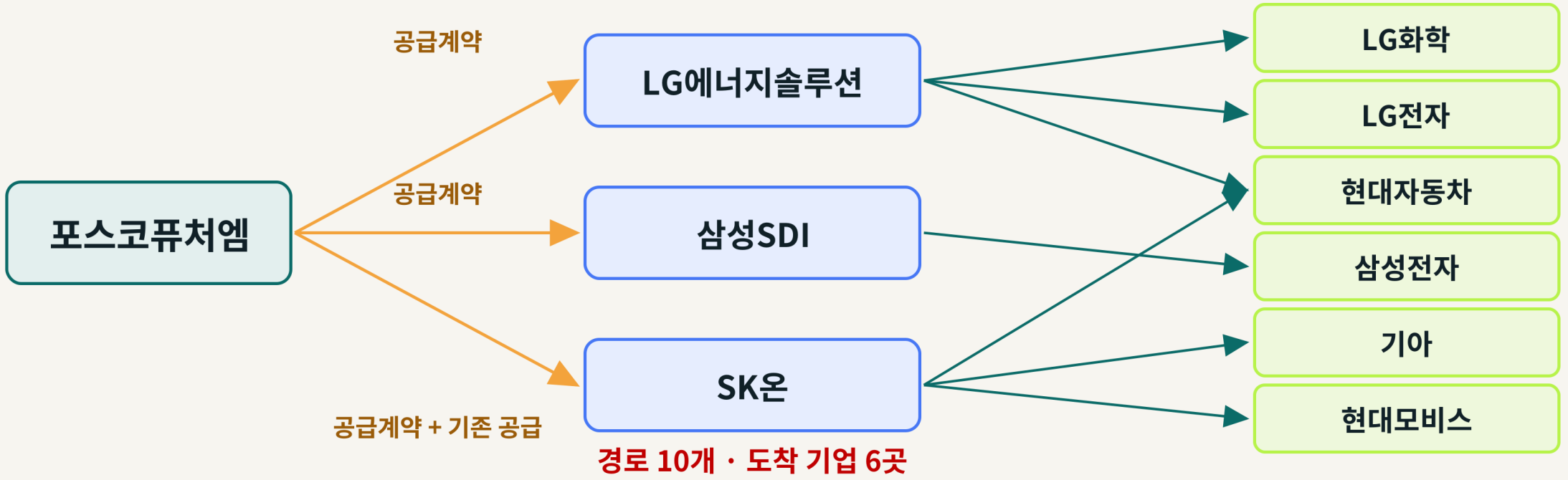
수요자	관계 구분	품목	자료일	진척상태
LG에너지솔루션	AGREEMENT	하이니켈 NCM·NCMA 양극재	2023-04-26	contract_reported
삼성SDI	AGREEMENT	LFP 양극재	2026-10-06	contract_signed
SK온	AGREEMENT	LFP 양극재	2026-09-22	contract_signed
SK온	SUPPLIES_TO	음극재	2026-09-22	existing_supply

AGREEMENT = SUPPLY_AGREEMENT_WITH · existing_supply = reported_existing_supply의 줄임입니다

CSV 계산

포스코퓨처엠의 2홉 경로는 10개이고 도착 기업은 6곳입니다

queries/09 — 경유 기업은 배터리 셀 3사뿐, 포스코(계열)는 다른 관계가 없어 경로를 만들지 않는다



SK온 경유 경로가 두 배인 것은 SK온까지 관계가 두 건(rel_015 · rel_016)이기 때문입니다

CSV 계산

2홉 경로 하나를 고르면 두 관계의 시점과 성격이 다릅니다

예시 경로: 포스코퓨처엠 —공급계약→ SK온 ←JV 계획— 현대자동차

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 **P10** P11 P12 P13 P14

관계 1 · rel_015

LFP 양극재 공급계약

- 2026-09-22 발표
- 2027~2029년 공급 예정

관계 2 · rel_026

북미 배터리셀 JV 계획

- 2023-04-25 이사회 승인
- 가동·출자 구조 미검증

암시하지 않는 것

전파의 근거 없음

- 현대차의 소재 사용
- 물량·매출 전파
- JV 공장 가동

3년 떨어진 두 발표를 하나의 공급 경로로 이어 읽으면 오답입니다

CSV 계산

현대자동차와 현대모비스 사이에는 근거 있는 관계가 두 건입니다

P10 모범답안 — (B) 방향은 두 건 모두 현대차 → 현대모비스

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 **P10** P11 P12 P13 P14

(A) 타입	(C) 주제	(D) 발표일	(E) 상태
AFFILIATED_WITH	현대차그룹	미상	group_membership
CO_MEMBER_OF_CONSORTIUM	MBD 컨소시엄	2023-04-07	consortium_membership

(F) 1차 출처: hyundai.com 계열사 목록 · 뉴스룸 detail/0000000228

※ 포스코퓨처엠–현대자동차를 고르면 직접 관계는 0건 — '미확인'으로 적고 2홉 경로만 언급한 답이 정답이다

같은 그룹에서 '모비스 지분 ○%'를 추정해 적은 답안은 0점 처리합니다

CSV 계산

과제 A는 계약 3건과 기존 공급 1건을 분리해 다시 씁니다

원문: “포스코퓨처엠과 배터리 회사가 연결되어 있으니 현재 실제로 모든 계약 물량을 납품하고 있다.”

모범 수정문 (5줄)

1. 포스코퓨처엠은 LG에너지솔루션(2023-04) · 삼성SDI(2026-10) · SK온(2026-09)과 양극재 공급계약을 발표했다.
2. 이는 계약 체결 사실이며, 납품 개시 시점과 물량은 별도 근거가 필요하다.
3. SK온 LFP 양극재 계약은 2027~2029년 공급 예정으로 발표됐다.
4. 기존 공급이 보도된 것은 SK온 음극재 1건(SUPPLIES_TO)이며 현재 물량은 미확인이다.
5. 따라서 '모든 계약 물량을 현재 납품 중'이라는 문장은 근거가 없다.

채점: 증거 유형 · 계약 체결 · 공급 이행 세 단어가 모두 구분되면 만점입니다

CSV 계산

과제 C의 모범 질의는 63쌍을 돌려줍니다

질문: 같은 차량 시스템에 연결되지만 공급계약·납품 관계는 없는 기업 쌍은?

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 **P10** P11 P12 P13 P14

모범 Cypher (읽기 전용)

```
MATCH (a:KAutoCompany)-[:HAS_CAPABILITY]->(:KAutoCapability)-[:ENABLES_SYSTEM]->(s:KAutoSystem)
      <-[:ENABLES_SYSTEM]-(:KAutoCapability)<-[:HAS_CAPABILITY]-(b:KAutoCompany)
WHERE a.id < b.id AND NOT EXISTS { (a)-[:SUPPLIES_TO|SUPPLY_AGREEMENT_WITH]-(b) }
RETURN DISTINCT s.name_ko AS 시스템, a.name_ko AS 기업A, b.name_ko AS 기업B;
```

기대 결과

63쌍 · 8개 시스템

최다

에너지 저장 33쌍

제외된 3쌍

포스코퓨처엠 × 셀 3사

a.id < b.id와 DISTINCT를 빼면 같은 쌍이 두세 번 나오는 것이 흔한 오류입니다

CSV 계산

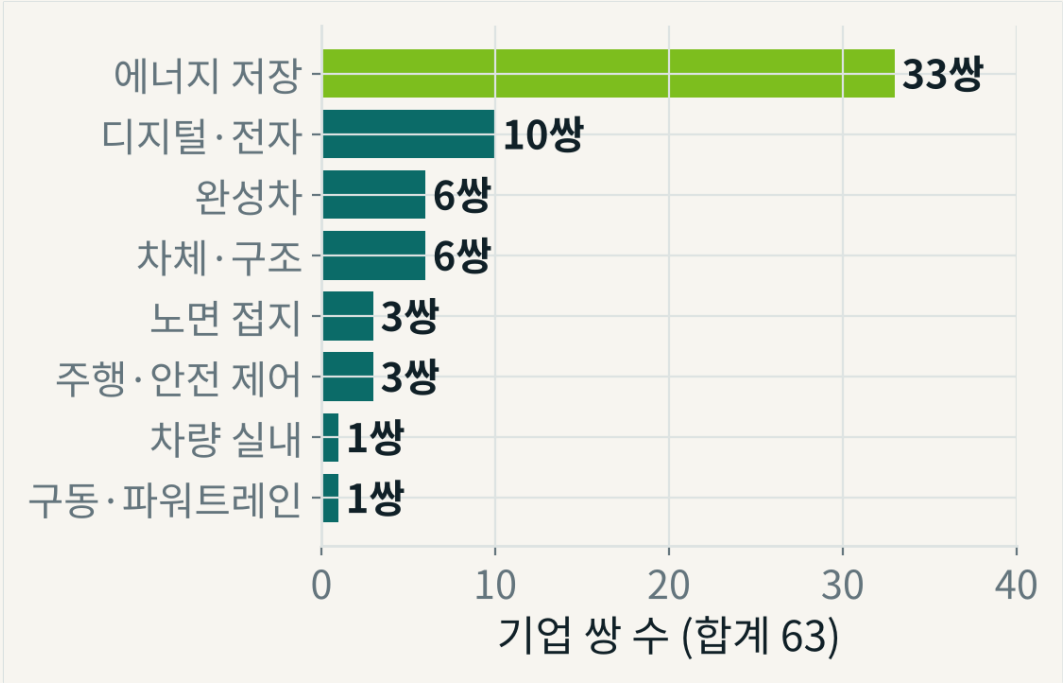
공급 관계 없는 같은 시스템 쌍은 에너지 저장에 몰려 있습니다

과제 C 모범 질의 결과를 시스템별로 센 값 — 해석 문장까지 써야 과제 C 완성이다

모범 해석

검증할 후보

- EV 소재·셀 33쌍
- 대부분 경쟁·협력 후보
- 납품은 미검증



이 목록은 '조사할 가설 목록'이지 '거래가 없다는 증명'이 아닙니다

04

MCP · 진단 · 채점

P11 ~ P14 · D1 ~ D6 — 등록과 호출을 가르고, 함정 두 개를 미리 안다

예시

claude mcp list에서 이름 · 두 절대 경로 · 연결 상태를 봅니다

macOS 예시 — 표시 형식은 Claude Code 버전마다 조금씩 다르다

claude mcp list

```
neo4j-kauto: /Users/kim/kauto_kg_mvp/.venv/bin/python
             /Users/kim/kauto_kg_mvp/scripts/mcp_launcher.py - ✓ Connected
```

통과

✓ Connected

실패 → D5

× Failed to connect

Windows 경로 예

\Scripts\python.exe

mcp_launcher.py도 common.py를 쓰므로 .env 비밀번호가 없으면 연결 단계에서 실패합니다

CSV 계산

P12 네 질문의 값은 앞서 계산한 기대값과 같아야 합니다

MCP 도구 이름은 Neo4j MCP 버전마다 다르다 — 값과 '사용한 도구명 표기'를 함께 채점한다

질문	기대 응답
(1) 노드 레이블	KAuto 6종 + 공용 레이블 5종 (Company 등)
(2) 기업 노드 개수	30
(3) 관계 타입별 개수	C1과 같은 8행 · 합계 42
(4) 현대모비스 역량	섀시 모듈 · 제동 시스템 · ADAS·센서 · 배터리 시스템

값이 맞아도 Python으로 대신 조회했다면 MCP 항목은 FAIL입니다

코드 확인

read-only 여부는 쓰기를 시도하지 말고 설정으로 확인합니다

Claude에게 '어디서 read-only가 강제되는지 보여 달라'고 묻는다 — v1.2 실행기 기준

scripts/mcp_launcher.py (v1.2 · 발췌)

```
env.update({
  'NEO4J_MCP_URI': cfg['uri'],
  'NEO4J_MCP_PASSWORD': cfg['password'],
  'NEO4J_MCP_READ_ONLY': 'true',
  'NEO4J_MCP_TELEMETRY': 'false', ...})
subprocess.call([sys.executable,
  '-m', 'neo4j_mcp_server',
  '--read-only', 'true'], env=env)
```

정답 위치 1

mcp_launcher.py가 환경변수와 --read-only 두 곳에서 고정

참고

.env의 NEO4J_READ_ONLY는 실행기가 무시하고 true로 덮는다

비밀번호는 환경변수로만 넘기고 claude mcp add 인자에 넣지 않는 것도 확인합니다

예시

D1~D3은 '무엇이 없는가'를 가르는 확인 명령부터 받습니다

진단 응답의 정답 형식 — 증상 → 판별 명령 → 기존 데이터를 지키는 해법

진단	판별 명령	안전한 해법
D1 claude 없음	where claude · which claude	공식 설치 또는 PATH 추가
D2 Docker 실패	docker info · Desktop 상태	Desktop 실행 · WSL2 확인
D3 인증 실패	docker volume ls	원래 비밀번호 · 볼륨 보존

판별 명령 없이 바로 재설치를 권하는 응답은 다시 요청합니다

예시

D4~D6은 경로와 개수를 대조해 원인을 좁힙니다

세 진단 모두 읽기 명령만으로 원인을 찾을 수 있다

진단	판별 명령	안전한 해법
D4 ModuleNotFound	sys.executable 경로 비교	.venv Python 직접 호출
D5 MCP 호출 안 됨	claude mcp get neo4j-kauto · /mcp	절대 경로 · .env 위치 확인
D6 42건이 아님	C1 · r.project별 count	차이 목록 → 승인 후 조치

D6에서 MATCH (n) DETACH DELETE n을 제안하면 즉시 중단시킵니다

실측

P13의 개선 대상은 이미 있는 항목을 빼면 둘뿐입니다

실측 HTML의 관계 카드에는 주제 · 발표일 · 상태 · 요약 · 출처 링크가 이미 표시된다

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 P10 P11 P12 **P13** P14

HTML 화면 — 카드에 두 항목 추가

- relation_type 원어
- reviewed_on 검토일
- 기존 항목은 유지

render_html.py 수정

Neo4j Cypher — C7에 열 추가

- r.reviewed_on
- r.evidence_level
- 읽기 전용 유지

queries 파일 수정

테스트 계획: 단위 테스트 6건 · HTML 재생성 · 42건 카드 표시 확인

코드 확인

P14를 그대로 하면 기존 테스트 두 개가 FAIL합니다

tests/test_dataset.py의 고정 조건 — 강사가 미리 알고 수강생에게 선택지를 주어야 한다

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 P10 P11 P12 P13 **P14**

신규 기업 1곳 추가 — 기업 수 고정 검사

- assertEqual(len, 30)
- 31개면 FAIL
- 테스트 수정은 승인 후

관계 추가만 권장

새 관계 1건 추가 — 발표일 상한 검사

- 발표일 ≤ 2026-10-19
- 그 뒤 발표면 FAIL
- 상한 갱신은 승인 후

새 ID는 rel_043

FAIL을 없애려고 테스트부터 고치는 응답은 순서가 틀린 것입니다

P14의 모범 진행은 여섯 단계를 이 순서로 밟습니다

각 단계의 결과를 보고받고 다음 단계로 — diff 승인 전에는 파일이 바뀌지 않아야 한다

P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 P10 P11 P12 P13 **P14**



※ verify_neo4j.py의 기대값은 CSV에서 다시 계산하므로 관계 1건을 추가하면 기대값 43으로 자동 갱신된다

출처를 못 찾으면 1단계에서 멈추는 것이 정답입니다

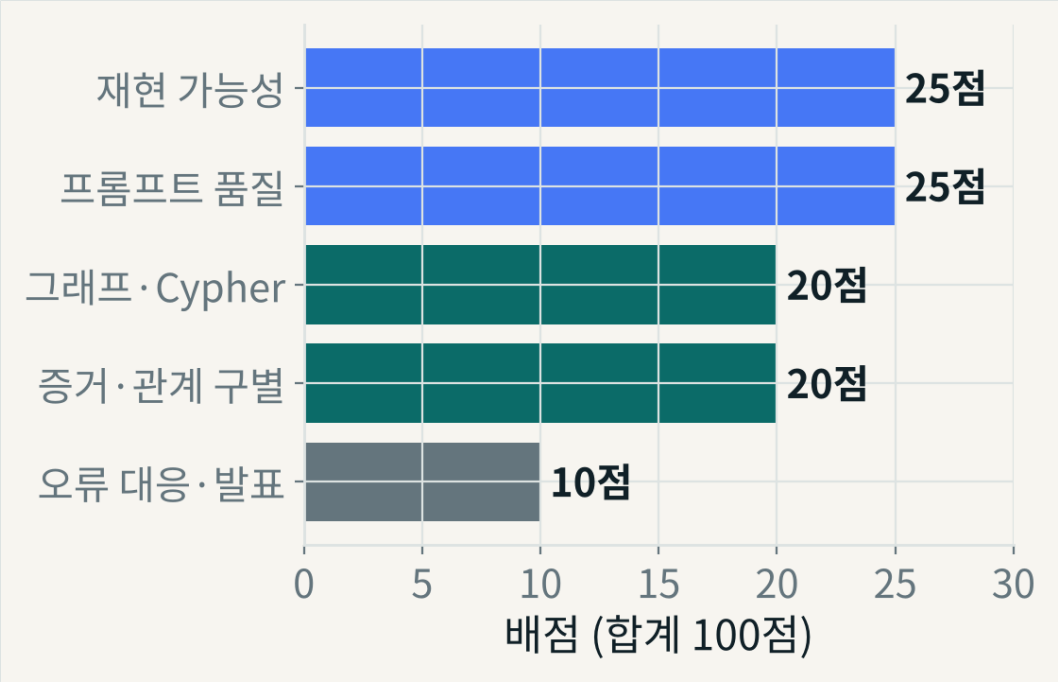
채점은 다섯 항목마다 '증거가 보고서에 있는가'로 합니다

권장 평가표 100점 — 결과 화면만 있고 명령·로그가 없으면 재현 가능성 점수를 주지 않는다

감점 사례

자주 보는 감점

- 로그 없는 '완료'
- 계약을 납품으로
- Python으로 MCP 대체



실패를 만나고 원인→수정→재시험을 남긴 보고서에 보너스를 줍니다

예시

모범 student_report.md는 확인된 사실과 미확인 사실을 나눠 적습니다

선택 관계: 포스코퓨처엠 → SK온 (rel_015) — 비밀번호·토큰은 없어야 한다

student_report.md (발췌)

```
## 실제 실행 명령과 결과
- .venv/bin/python -m unittest discover -s tests -v / 프로젝트 폴더 / PASS / Ran 6 tests OK
- .venv/bin/python scripts/verify_neo4j.py / 프로젝트 폴더 / PASS / company_company_edges: 42
## 선택한 기업 간 관계 1건
- 포스코퓨처엠 → SUPPLY_AGREEMENT_WITH → SK온 / 2026-09-22 / contract_signed / poscofuturem.com
- 확인된 사실: LFP 양극재 공급계약 발표, 2027~2029년 공급 예정
- 아직 증명되지 않은 사실: 납품 개시 여부, 연도별 물량, 계약 금액의 매출 인식 시점
```

'아직 증명되지 않은 사실' 칸이 비어 있으면 증거·관계 구별 점수를 깎습니다

실측과 계산을 구분하고

기대값과 대조하고

근거가 없으면 미확인으로 둔다

정답은 숫자보다
판정의 근거에 있습니다.

※ CSV 계산값은 시드 데이터(출처 검토일 2026-10-19) 기준. Docker·Neo4j·MCP 화면은 이 자료 제작 환경에서 실행하지 않았으며 '예시'로 표시했다.